

Aula 15 - Estruturas de Dados (Set, Map e Heap)

Nessa aula, vemos algumas estruturas de dados um pouco mais avançadas mais incomuns que as mais básicas, mas que podem vir a ser úteis.

Set (Conjunto)

Uma estrutura que consiste em uma coleção de elementos:

- Únicos → em um set, não existem elementos duplicados
- Ordenados → por padrão eles são ordenados internamente em tempo de inserção

Na prática, um set é muito útil para remover duplicatas, consultas rápidas de existência, manter números ordenados, etc;

Em um set, também não existe o acesso aos elementos via índice (como nos vetores), no caso, temos funções de acesso, como nas pilhas e filas.

Operações

- `set<int> s` → declara um set de inteiros chamado *s*
- `set<int, greater<int>> s` → declara um set de inteiros que mantém ordenado de forma decrescente
- `s.insert(x)` → insere o elemento *x* no set *s*
- `s.erase(x)` → remove o elemento *x* do set, se não existir, não faz nada
- `s.count(x)` → retorna 0 ou 1, verifica se *x* está no set
- `s.begin()` → retorna um ponteiro que aponta para o início do set
- `s.end()` → retorna um ponteiro que aponta para o fim do set
- `s.empty()` → verifica se o set está vazio ou não
- `s.size()` → retorna o tamanho do set (quantidade de elementos)

- `s.clear()` → remove todos os elementos do set
- `s.find(x)` → retorna um iterator apontando para x , ou `end()` se não encontrar
- `s.lower_bound(x)` → retorna o primeiro elemento $\geq x$, retorna `end()` se não encontrar
- `s.upper_bound(x)` → retorna o primeiro elemento $> x$, retorna `end()` se não encontrar

Iterando por um set

Existem duas formas de iterar por um set, a primeira é usando um range-based `for`:

```
set<int> s;

for(int x : s){
    cout << x;
}
```

A segunda é fazendo uso de **iteradores**.

```
for(auto it = s.begin(); it != s.end(); it++){
    cout << *it << " ";
}
```

Pegando o menor/maior elemento de um set

Menor

```
cout << *s.begin();
```

Maior

```
auto it = prev(s.end());  
cout << *it;
```

Sucessor e predecessor

Sucessor de x

```
auto it = s.upper_bound(x);  
  
if(it != s.end())  
    cout << *it;
```

Predecessor de x

```
auto it = s.lower_bound(x);  
  
if(it != s.begin()) {  
    it--;  
    cout << *it;  
}
```

Remover duplicatas de um vetor

```
vector<int> v = {1, 5, 2, 2, 1, 9};  
  
set<int> s(v.begin(), v.end());  
  
vector<int> v(s.begin(), s.end());
```

Multiset

Um parente do Set comum, funciona da exata mesma forma, mas permite duplicatas. As funções básicas são exatamente iguais, mas temos algumas pequenas diferenças.

Removendo elementos de um Multiset

Se usarmos o `erase(x)` tal qual no Set comum, estaremos apagando todo mundo que tem valor = x . Se isso não for o desejado, precisamos remover via iterador:

```
auto it = ms.find(5);

ms.erase(it); // se tínhamos três 5's, agora temos só dois
ms.erase(5); // agora não temos mais nenhum
```

Intervalo igual

A função `equal_range` acha o início de um intervalo de um Multiset que contém elementos repetidos. Por exemplo, se um Multiset era $\{1, 2, 2, 3, 3, 3, 4, 5\}$, podemos achar o `equal_range(3)` que consiste no início do intervalo em que todo mundo é igual a 3.

```
auto [ini, fim] = ms.equal_range(3);
```

Unordered Set

Mais um parente do Set comum, essa versão não mantém a ordem dos elementos, e, portanto, perde muitas das suas utilidades. A grande vantagem dessa versão está na absurda eficiência nas quatro operações básicas: `insert`, `erase`, `find`, e `count`, que são todas feitas em $O(1)$ ao invés de em $O(\log n)$ como no Set padrão.

Map

Uma estrutura de dados de acesso por índice, tal qual os vetores, mas que não amarra os índices a números inteiros. Ou seja, na prática, o Map guarda pares de

elementos, o primeiro elemento é usado como chave/índice e o segundo elemento é usado como valor.

```
map<string, int> idade; // mapeia strings para inteiros
```

Para inserir elementos, podemos simplesmente dar um valor a uma chave que ainda não existe:

```
idade["Gabriel"] = 19;
```

Ou podemos inserir um `pair<string, int>`:

```
idade.insert(make_pair("Gabriel", 19))  
// ou com um par já existente  
pair<string, int> p;  
p.first = "Gabriel";  
p.second = 19;  
  
idade.insert(p);
```

Para acessar, fazemos tal qual em um vetor, usando uma string como chave:

```
cout << idade["Gabriel"];
```

Podemos usar um `find(x)` aqui também, da mesma forma como em Sets.

```
if(idade.find("Gabriel") != idade.end()) cout << idade["Gabriel"];  
else cout << "Idade não encontrada";
```

E, para percorrer um Map, fazemos da mesma forma como em Sets. Usando range-based loops ou iterators:

```
for(auto p : idade)
```

```
for(auto it = idade.begin(); it != idade.end(); it++)
```

Unordered Map

De forma extremamente similar ao Set e sua contraparte desordenada, também temos um unordered map que perde as propriedades vindas da ordenação, mas ganha bastante eficiência em operações de inserção, remoção e verificação de existência.

Priority Queue

É uma versão de fila cuja função `top()` retorna sempre o elemento de **maior prioridade**, isto é, via de regra, o maior valor dentro da fila naquele momento. Essa estrutura é muito comumente chamada de **heap**, podendo ser uma **max heap** se retorna sempre o **maior valor** ou uma **min heap** se retorna sempre o **menor valor**.

Via de regra, declarar e utilizar uma heap é extremamente similar ao que faríamos com uma fila/pilha, com exceção da declaração.

A declaração padrão da heap:

```
priority_queue<int> pq;
```

Fornece sempre uma max heap. Se quiséssemos que essa heap fosse mínima, teríamos que declará-la de uma forma diferente. A declaração de uma heap segue o formato:

```
priority_queue<
    TIPO,
    CONTAINER,
    COMPARADOR
>
```

Em que:

- TIPO → se refere ao tipo que estamos guardando

- CONTAINER → se refere ao tipo de container interno que a heap usa, quase sempre nós ficamos alheios a isso, mas em alguns usos da heap (como Dijkstra) precisamos mexer nisso
- COMPARADOR → é a função de comparação que entregamos para a heap descobrir quem é o elemento de "maior prioridade"

Em Dijkstra, nós usamos:

```
priority_queue<
    pair<int, int>,
    vector<pair<int, int>,
    greater<pair<int, int>>
>
```

Já que queremos armazenar pares, precisamos fornecer como tipo da heap um par, como container um vetor de pares e como comparador a função `greater<int>`, que determina ordenação crescente ao invés da padrão decrescente.

OBS.: essa função também pode ser usada no `sort` em vetores para alterar o tipo de ordenação!